

```
In [1]: # # This mounts your Google Drive to the Colab VM.
# from google.colab import drive
# drive.mount('/content/drive')

# # TODO: Enter the foldername in your Drive where you have saved the unzipped
# # assignment folder, e.g. 'cs6353/assignments/assignment2/'
# FOLDERNAME = 'assignment2'
# assert FOLDERNAME is not None, "[!] Enter the foldername."

# # Now that we've mounted your Drive, this ensures that
# # the Python interpreter of the Colab VM can load
# # python files from within it.
# import sys
# sys.path.append('/content/drive/My Drive/{ }'.format(FOLDERNAME))

# ----- Laptop -----
# %cd "/home/jared/Desktop/Fall2025/CS-6353/Homeworks/assignment2/cs6353/datasets/"
# !bash get_datasets.sh
# %cd "/home/jared/Desktop/Fall2025/CS-6353/Homeworks/assignment2"
# ----- Laptop -----

# ----- Desktop -----
%cd "/home/jared/CS-6353/Homeworks/assignment2/cs6353/datasets/"
!bash get_datasets.sh
%cd "/home/jared/CS-6353/Homeworks/assignment2/"
# ----- Desktop -----

# Install requirements from colab_requirements.txt
# TODO: Please change your path below to the colab_requirements.txt file

# ----- Desktop -----
! python -m pip install -r "/home/jared/CS-6353/Homeworks/assignment2/colab_require
# ----- Desktop -----

# ----- Laptop -----
# ! python -m pip install -r "/home/jared/Desktop/Fall2025/CS-6353/Homeworks/assign
# ----- Laptop -----
```

```
/home/jared/CS-6353/Homeworks/assignment2/cs6353/datasets
--2025-10-01 18:59:01-- http://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz
Resolving www.cs.toronto.edu (www.cs.toronto.edu)... 128.100.3.30
Connecting to www.cs.toronto.edu (www.cs.toronto.edu)|128.100.3.30|:80... connected.
HTTP request sent, awaiting response... 200 OK
Length: 170498071 (163M) [application/x-gzip]
Saving to: 'cifar-10-python.tar.gz'
```

```
cifar-10-python.tar 100%[=====>] 162.60M 23.9MB/s in 5.2s
```

```
2025-10-01 18:59:07 (31.5 MB/s) - 'cifar-10-python.tar.gz' saved [170498071/170498071]
```

```
cifar-10-batches-py/
cifar-10-batches-py/data_batch_4
cifar-10-batches-py/readme.html
cifar-10-batches-py/test_batch
cifar-10-batches-py/data_batch_3
cifar-10-batches-py/batches.meta
cifar-10-batches-py/data_batch_2
cifar-10-batches-py/data_batch_5
cifar-10-batches-py/data_batch_1
/home/jared/CS-6353/Homeworks/assignment2
Requirement already satisfied: anyio==3.7.1 in /home/jared/anaconda3/envs/CS6353/lib/python3.10/site-packages (from -r /home/jared/CS-6353/Homeworks/assignment2/colab_requirements.txt (line 1)) (3.7.1)
Requirement already satisfied: appnope==0.1.3 in /home/jared/anaconda3/envs/CS6353/lib/python3.10/site-packages (from -r /home/jared/CS-6353/Homeworks/assignment2/colab_requirements.txt (line 2)) (0.1.3)
Requirement already satisfied: argon2-cffi==23.1.0 in /home/jared/anaconda3/envs/CS6353/lib/python3.10/site-packages (from -r /home/jared/CS-6353/Homeworks/assignment2/colab_requirements.txt (line 3)) (23.1.0)
Requirement already satisfied: argon2-cffi-bindings==21.2.0 in /home/jared/anaconda3/envs/CS6353/lib/python3.10/site-packages (from -r /home/jared/CS-6353/Homeworks/assignment2/colab_requirements.txt (line 4)) (21.2.0)
Requirement already satisfied: arrow==1.2.3 in /home/jared/anaconda3/envs/CS6353/lib/python3.10/site-packages (from -r /home/jared/CS-6353/Homeworks/assignment2/colab_requirements.txt (line 5)) (1.2.3)
Requirement already satisfied: asttokens==2.2.1 in /home/jared/anaconda3/envs/CS6353/lib/python3.10/site-packages (from -r /home/jared/CS-6353/Homeworks/assignment2/colab_requirements.txt (line 6)) (2.2.1)
Requirement already satisfied: async-lru==2.0.4 in /home/jared/anaconda3/envs/CS6353/lib/python3.10/site-packages (from -r /home/jared/CS-6353/Homeworks/assignment2/colab_requirements.txt (line 7)) (2.0.4)
Requirement already satisfied: attrs==23.1.0 in /home/jared/anaconda3/envs/CS6353/lib/python3.10/site-packages (from -r /home/jared/CS-6353/Homeworks/assignment2/colab_requirements.txt (line 8)) (23.1.0)
Requirement already satisfied: Babel==2.12.1 in /home/jared/anaconda3/envs/CS6353/lib/python3.10/site-packages (from -r /home/jared/CS-6353/Homeworks/assignment2/colab_requirements.txt (line 9)) (2.12.1)
Requirement already satisfied: backcall==0.2.0 in /home/jared/anaconda3/envs/CS6353/lib/python3.10/site-packages (from -r /home/jared/CS-6353/Homeworks/assignment2/colab_requirements.txt (line 10)) (0.2.0)
Requirement already satisfied: beautifulsoup4==4.12.2 in /home/jared/anaconda3/envs/CS6353/lib/python3.10/site-packages (from -r /home/jared/CS-6353/Homeworks/assignment2/colab_requirements.txt (line 11)) (4.12.2)
```

Requirement already satisfied: bleach==6.0.0 in /home/jared/anaconda3/envs/CS6353/lib/python3.10/site-packages (from -r /home/jared/CS-6353/Homeworks/assignment2/colab_requirements.txt (line 12)) (6.0.0)

Requirement already satisfied: certifi==2023.7.22 in /home/jared/anaconda3/envs/CS6353/lib/python3.10/site-packages (from -r /home/jared/CS-6353/Homeworks/assignment2/colab_requirements.txt (line 13)) (2023.7.22)

Requirement already satisfied: cffi==1.15.1 in /home/jared/anaconda3/envs/CS6353/lib/python3.10/site-packages (from -r /home/jared/CS-6353/Homeworks/assignment2/colab_requirements.txt (line 14)) (1.15.1)

Requirement already satisfied: charset-normalizer==3.2.0 in /home/jared/anaconda3/envs/CS6353/lib/python3.10/site-packages (from -r /home/jared/CS-6353/Homeworks/assignment2/colab_requirements.txt (line 15)) (3.2.0)

Requirement already satisfied: comm==0.1.4 in /home/jared/anaconda3/envs/CS6353/lib/python3.10/site-packages (from -r /home/jared/CS-6353/Homeworks/assignment2/colab_requirements.txt (line 16)) (0.1.4)

Requirement already satisfied: contourpy==1.1.0 in /home/jared/anaconda3/envs/CS6353/lib/python3.10/site-packages (from -r /home/jared/CS-6353/Homeworks/assignment2/colab_requirements.txt (line 17)) (1.1.0)

Requirement already satisfied: cycler==0.11.0 in /home/jared/anaconda3/envs/CS6353/lib/python3.10/site-packages (from -r /home/jared/CS-6353/Homeworks/assignment2/colab_requirements.txt (line 18)) (0.11.0)

Requirement already satisfied: debugpy==1.6.7.post1 in /home/jared/anaconda3/envs/CS6353/lib/python3.10/site-packages (from -r /home/jared/CS-6353/Homeworks/assignment2/colab_requirements.txt (line 19)) (1.6.7.post1)

Requirement already satisfied: decorator<=5.0 in /home/jared/anaconda3/envs/CS6353/lib/python3.10/site-packages (from -r /home/jared/CS-6353/Homeworks/assignment2/colab_requirements.txt (line 20)) (4.4.2)

Requirement already satisfied: defusedxml==0.7.1 in /home/jared/anaconda3/envs/CS6353/lib/python3.10/site-packages (from -r /home/jared/CS-6353/Homeworks/assignment2/colab_requirements.txt (line 21)) (0.7.1)

Requirement already satisfied: executing==1.2.0 in /home/jared/anaconda3/envs/CS6353/lib/python3.10/site-packages (from -r /home/jared/CS-6353/Homeworks/assignment2/colab_requirements.txt (line 22)) (1.2.0)

Requirement already satisfied: fastjsonschema==2.18.0 in /home/jared/anaconda3/envs/CS6353/lib/python3.10/site-packages (from -r /home/jared/CS-6353/Homeworks/assignment2/colab_requirements.txt (line 23)) (2.18.0)

Requirement already satisfied: fonttools==4.42.1 in /home/jared/anaconda3/envs/CS6353/lib/python3.10/site-packages (from -r /home/jared/CS-6353/Homeworks/assignment2/colab_requirements.txt (line 24)) (4.42.1)

Requirement already satisfied: fqdn==1.5.1 in /home/jared/anaconda3/envs/CS6353/lib/python3.10/site-packages (from -r /home/jared/CS-6353/Homeworks/assignment2/colab_requirements.txt (line 25)) (1.5.1)

Requirement already satisfied: idna==3.4 in /home/jared/anaconda3/envs/CS6353/lib/python3.10/site-packages (from -r /home/jared/CS-6353/Homeworks/assignment2/colab_requirements.txt (line 26)) (3.4)

Requirement already satisfied: imageio==2.31.1 in /home/jared/anaconda3/envs/CS6353/lib/python3.10/site-packages (from -r /home/jared/CS-6353/Homeworks/assignment2/colab_requirements.txt (line 27)) (2.31.1)

Requirement already satisfied: ipykernel<=5.5.6 in /home/jared/anaconda3/envs/CS6353/lib/python3.10/site-packages (from -r /home/jared/CS-6353/Homeworks/assignment2/colab_requirements.txt (line 28)) (5.5.6)

Requirement already satisfied: ipython<=7.34.0 in /home/jared/anaconda3/envs/CS6353/lib/python3.10/site-packages (from -r /home/jared/CS-6353/Homeworks/assignment2/colab_requirements.txt (line 29)) (7.34.0)

Requirement already satisfied: isoduration==20.11.0 in /home/jared/anaconda3/envs/CS6353/lib/python3.10/site-packages (from -r /home/jared/CS-6353/Homeworks/assignment

```
2/colab_requirements.txt (line 30)) (20.11.0)
Requirement already satisfied: jedi==0.19.0 in /home/jared/anaconda3/envs/CS6353/lib/python3.10/site-packages (from -r /home/jared/CS-6353/Homeworks/assignment2/colab_requirements.txt (line 31)) (0.19.0)
Requirement already satisfied: Jinja2==3.1.2 in /home/jared/anaconda3/envs/CS6353/lib/python3.10/site-packages (from -r /home/jared/CS-6353/Homeworks/assignment2/colab_requirements.txt (line 32)) (3.1.2)
Requirement already satisfied: json5==0.9.14 in /home/jared/anaconda3/envs/CS6353/lib/python3.10/site-packages (from -r /home/jared/CS-6353/Homeworks/assignment2/colab_requirements.txt (line 33)) (0.9.14)
Requirement already satisfied: jsonpointer==2.4 in /home/jared/anaconda3/envs/CS6353/lib/python3.10/site-packages (from -r /home/jared/CS-6353/Homeworks/assignment2/colab_requirements.txt (line 34)) (2.4)
Requirement already satisfied: jsonschema==4.19.0 in /home/jared/anaconda3/envs/CS6353/lib/python3.10/site-packages (from -r /home/jared/CS-6353/Homeworks/assignment2/colab_requirements.txt (line 35)) (4.19.0)
Requirement already satisfied: jsonschema-specifications==2023.7.1 in /home/jared/anaconda3/envs/CS6353/lib/python3.10/site-packages (from -r /home/jared/CS-6353/Homeworks/assignment2/colab_requirements.txt (line 36)) (2023.7.1)
Requirement already satisfied: jupyter-events==0.7.0 in /home/jared/anaconda3/envs/CS6353/lib/python3.10/site-packages (from -r /home/jared/CS-6353/Homeworks/assignment2/colab_requirements.txt (line 37)) (0.7.0)
Requirement already satisfied: jupyter-lsp==2.2.0 in /home/jared/anaconda3/envs/CS6353/lib/python3.10/site-packages (from -r /home/jared/CS-6353/Homeworks/assignment2/colab_requirements.txt (line 38)) (2.2.0)
Requirement already satisfied: jupyter_client<8.0 in /home/jared/anaconda3/envs/CS6353/lib/python3.10/site-packages (from -r /home/jared/CS-6353/Homeworks/assignment2/colab_requirements.txt (line 39)) (7.4.9)
Requirement already satisfied: jupyter_core==5.3.1 in /home/jared/anaconda3/envs/CS6353/lib/python3.10/site-packages (from -r /home/jared/CS-6353/Homeworks/assignment2/colab_requirements.txt (line 40)) (5.3.1)
Requirement already satisfied: jupyter_server==2.7.2 in /home/jared/anaconda3/envs/CS6353/lib/python3.10/site-packages (from -r /home/jared/CS-6353/Homeworks/assignment2/colab_requirements.txt (line 41)) (2.7.2)
Requirement already satisfied: jupyter_server_terminals==0.4.4 in /home/jared/anaconda3/envs/CS6353/lib/python3.10/site-packages (from -r /home/jared/CS-6353/Homeworks/assignment2/colab_requirements.txt (line 42)) (0.4.4)
Requirement already satisfied: jupyterlab==4.0.5 in /home/jared/anaconda3/envs/CS6353/lib/python3.10/site-packages (from -r /home/jared/CS-6353/Homeworks/assignment2/colab_requirements.txt (line 43)) (4.0.5)
Requirement already satisfied: jupyterlab-pygments==0.2.2 in /home/jared/anaconda3/envs/CS6353/lib/python3.10/site-packages (from -r /home/jared/CS-6353/Homeworks/assignment2/colab_requirements.txt (line 44)) (0.2.2)
Requirement already satisfied: jupyterlab_server==2.24.0 in /home/jared/anaconda3/envs/CS6353/lib/python3.10/site-packages (from -r /home/jared/CS-6353/Homeworks/assignment2/colab_requirements.txt (line 45)) (2.24.0)
Requirement already satisfied: kiwisolver==1.4.5 in /home/jared/anaconda3/envs/CS6353/lib/python3.10/site-packages (from -r /home/jared/CS-6353/Homeworks/assignment2/colab_requirements.txt (line 46)) (1.4.5)
Requirement already satisfied: MarkupSafe==2.1.3 in /home/jared/anaconda3/envs/CS6353/lib/python3.10/site-packages (from -r /home/jared/CS-6353/Homeworks/assignment2/colab_requirements.txt (line 47)) (2.1.3)
Requirement already satisfied: matplotlib==3.7.2 in /home/jared/anaconda3/envs/CS6353/lib/python3.10/site-packages (from -r /home/jared/CS-6353/Homeworks/assignment2/colab_requirements.txt (line 48)) (3.7.2)
Requirement already satisfied: matplotlib-inline==0.1.6 in /home/jared/anaconda3/env
```

```
s/CS6353/lib/python3.10/site-packages (from -r /home/jared/CS-6353/Homeworks/assignment2/colab_requirements.txt (line 49)) (0.1.6)
Requirement already satisfied: mistune==3.0.1 in /home/jared/anaconda3/envs/CS6353/lib/python3.10/site-packages (from -r /home/jared/CS-6353/Homeworks/assignment2/colab_requirements.txt (line 50)) (3.0.1)
Requirement already satisfied: nbclient==0.8.0 in /home/jared/anaconda3/envs/CS6353/lib/python3.10/site-packages (from -r /home/jared/CS-6353/Homeworks/assignment2/colab_requirements.txt (line 51)) (0.8.0)
Requirement already satisfied: nbconvert==7.7.4 in /home/jared/anaconda3/envs/CS6353/lib/python3.10/site-packages (from -r /home/jared/CS-6353/Homeworks/assignment2/colab_requirements.txt (line 52)) (7.7.4)
Requirement already satisfied: nbformat==5.9.2 in /home/jared/anaconda3/envs/CS6353/lib/python3.10/site-packages (from -r /home/jared/CS-6353/Homeworks/assignment2/colab_requirements.txt (line 53)) (5.9.2)
Requirement already satisfied: nest-asyncio==1.5.7 in /home/jared/anaconda3/envs/CS6353/lib/python3.10/site-packages (from -r /home/jared/CS-6353/Homeworks/assignment2/colab_requirements.txt (line 54)) (1.5.7)
Requirement already satisfied: notebook_shim==0.2.3 in /home/jared/anaconda3/envs/CS6353/lib/python3.10/site-packages (from -r /home/jared/CS-6353/Homeworks/assignment2/colab_requirements.txt (line 55)) (0.2.3)
Requirement already satisfied: numpy<1.24,>=1.22 in /home/jared/anaconda3/envs/CS6353/lib/python3.10/site-packages (from -r /home/jared/CS-6353/Homeworks/assignment2/colab_requirements.txt (line 56)) (1.23.5)
Requirement already satisfied: overrides==7.4.0 in /home/jared/anaconda3/envs/CS6353/lib/python3.10/site-packages (from -r /home/jared/CS-6353/Homeworks/assignment2/colab_requirements.txt (line 57)) (7.4.0)
Requirement already satisfied: packaging==23.1 in /home/jared/anaconda3/envs/CS6353/lib/python3.10/site-packages (from -r /home/jared/CS-6353/Homeworks/assignment2/colab_requirements.txt (line 58)) (23.1)
Requirement already satisfied: pandas<=1.5.3 in /home/jared/anaconda3/envs/CS6353/lib/python3.10/site-packages (from -r /home/jared/CS-6353/Homeworks/assignment2/colab_requirements.txt (line 59)) (1.5.3)
Requirement already satisfied: pandocfilters==1.5.0 in /home/jared/anaconda3/envs/CS6353/lib/python3.10/site-packages (from -r /home/jared/CS-6353/Homeworks/assignment2/colab_requirements.txt (line 60)) (1.5.0)
Requirement already satisfied: parso==0.8.3 in /home/jared/anaconda3/envs/CS6353/lib/python3.10/site-packages (from -r /home/jared/CS-6353/Homeworks/assignment2/colab_requirements.txt (line 61)) (0.8.3)
Requirement already satisfied: pexpect==4.8.0 in /home/jared/anaconda3/envs/CS6353/lib/python3.10/site-packages (from -r /home/jared/CS-6353/Homeworks/assignment2/colab_requirements.txt (line 62)) (4.8.0)
Requirement already satisfied: pickleshare==0.7.5 in /home/jared/anaconda3/envs/CS6353/lib/python3.10/site-packages (from -r /home/jared/CS-6353/Homeworks/assignment2/colab_requirements.txt (line 63)) (0.7.5)
Requirement already satisfied: Pillow==10.0.0 in /home/jared/anaconda3/envs/CS6353/lib/python3.10/site-packages (from -r /home/jared/CS-6353/Homeworks/assignment2/colab_requirements.txt (line 64)) (10.0.0)
Requirement already satisfied: platformdirs==3.10.0 in /home/jared/anaconda3/envs/CS6353/lib/python3.10/site-packages (from -r /home/jared/CS-6353/Homeworks/assignment2/colab_requirements.txt (line 65)) (3.10.0)
Requirement already satisfied: prometheus-client==0.17.1 in /home/jared/anaconda3/envs/CS6353/lib/python3.10/site-packages (from -r /home/jared/CS-6353/Homeworks/assignment2/colab_requirements.txt (line 66)) (0.17.1)
Requirement already satisfied: prompt-toolkit==3.0.39 in /home/jared/anaconda3/envs/CS6353/lib/python3.10/site-packages (from -r /home/jared/CS-6353/Homeworks/assignment2/colab_requirements.txt (line 67)) (3.0.39)
```

Requirement already satisfied: psutil==5.9.5 in /home/jared/anaconda3/envs/CS6353/lib/python3.10/site-packages (from -r /home/jared/CS-6353/Homeworks/assignment2/colab_requirements.txt (line 68)) (5.9.5)

Requirement already satisfied: ptyprocess==0.7.0 in /home/jared/anaconda3/envs/CS6353/lib/python3.10/site-packages (from -r /home/jared/CS-6353/Homeworks/assignment2/colab_requirements.txt (line 69)) (0.7.0)

Requirement already satisfied: pure-eval==0.2.2 in /home/jared/anaconda3/envs/CS6353/lib/python3.10/site-packages (from -r /home/jared/CS-6353/Homeworks/assignment2/colab_requirements.txt (line 70)) (0.2.2)

Requirement already satisfied: pycparser==2.21 in /home/jared/anaconda3/envs/CS6353/lib/python3.10/site-packages (from -r /home/jared/CS-6353/Homeworks/assignment2/colab_requirements.txt (line 71)) (2.21)

Requirement already satisfied: Pygments==2.16.1 in /home/jared/anaconda3/envs/CS6353/lib/python3.10/site-packages (from -r /home/jared/CS-6353/Homeworks/assignment2/colab_requirements.txt (line 72)) (2.16.1)

Requirement already satisfied: pyparsing==3.0.9 in /home/jared/anaconda3/envs/CS6353/lib/python3.10/site-packages (from -r /home/jared/CS-6353/Homeworks/assignment2/colab_requirements.txt (line 73)) (3.0.9)

Requirement already satisfied: python-dateutil==2.8.2 in /home/jared/anaconda3/envs/CS6353/lib/python3.10/site-packages (from -r /home/jared/CS-6353/Homeworks/assignment2/colab_requirements.txt (line 74)) (2.8.2)

Requirement already satisfied: python-json-logger==2.0.7 in /home/jared/anaconda3/envs/CS6353/lib/python3.10/site-packages (from -r /home/jared/CS-6353/Homeworks/assignment2/colab_requirements.txt (line 75)) (2.0.7)

Requirement already satisfied: pytz==2023.3 in /home/jared/anaconda3/envs/CS6353/lib/python3.10/site-packages (from -r /home/jared/CS-6353/Homeworks/assignment2/colab_requirements.txt (line 76)) (2023.3)

Requirement already satisfied: PyYAML==6.0.1 in /home/jared/anaconda3/envs/CS6353/lib/python3.10/site-packages (from -r /home/jared/CS-6353/Homeworks/assignment2/colab_requirements.txt (line 77)) (6.0.1)

Requirement already satisfied: pyzmq<25 in /home/jared/anaconda3/envs/CS6353/lib/python3.10/site-packages (from -r /home/jared/CS-6353/Homeworks/assignment2/colab_requirements.txt (line 78)) (24.0.1)

Requirement already satisfied: referencing==0.30.2 in /home/jared/anaconda3/envs/CS6353/lib/python3.10/site-packages (from -r /home/jared/CS-6353/Homeworks/assignment2/colab_requirements.txt (line 79)) (0.30.2)

Requirement already satisfied: requests==2.31.0 in /home/jared/anaconda3/envs/CS6353/lib/python3.10/site-packages (from -r /home/jared/CS-6353/Homeworks/assignment2/colab_requirements.txt (line 80)) (2.31.0)

Requirement already satisfied: rfc3339-validator==0.1.4 in /home/jared/anaconda3/envs/CS6353/lib/python3.10/site-packages (from -r /home/jared/CS-6353/Homeworks/assignment2/colab_requirements.txt (line 81)) (0.1.4)

Requirement already satisfied: rfc3986-validator==0.1.1 in /home/jared/anaconda3/envs/CS6353/lib/python3.10/site-packages (from -r /home/jared/CS-6353/Homeworks/assignment2/colab_requirements.txt (line 82)) (0.1.1)

Requirement already satisfied: rpds-py==0.9.2 in /home/jared/anaconda3/envs/CS6353/lib/python3.10/site-packages (from -r /home/jared/CS-6353/Homeworks/assignment2/colab_requirements.txt (line 83)) (0.9.2)

Requirement already satisfied: scipy==1.11.2 in /home/jared/anaconda3/envs/CS6353/lib/python3.10/site-packages (from -r /home/jared/CS-6353/Homeworks/assignment2/colab_requirements.txt (line 84)) (1.11.2)

Requirement already satisfied: seaborn==0.12.2 in /home/jared/anaconda3/envs/CS6353/lib/python3.10/site-packages (from -r /home/jared/CS-6353/Homeworks/assignment2/colab_requirements.txt (line 85)) (0.12.2)

Requirement already satisfied: Send2Trash==1.8.2 in /home/jared/anaconda3/envs/CS6353/lib/python3.10/site-packages (from -r /home/jared/CS-6353/Homeworks/assignment2/colab_requirements.txt (line 86)) (1.8.2)

```
lab_requirements.txt (line 86)) (1.8.2)
Requirement already satisfied: six==1.16.0 in /home/jared/anaconda3/envs/CS6353/lib/
python3.10/site-packages (from -r /home/jared/CS-6353/Homeworks/assignment2/colab_re
quirements.txt (line 87)) (1.16.0)
Requirement already satisfied: sniffio==1.3.0 in /home/jared/anaconda3/envs/CS6353/l
ib/python3.10/site-packages (from -r /home/jared/CS-6353/Homeworks/assignment2/colab
_requirements.txt (line 88)) (1.3.0)
Requirement already satisfied: soupsieve==2.4.1 in /home/jared/anaconda3/envs/CS635
3/lib/python3.10/site-packages (from -r /home/jared/CS-6353/Homeworks/assignment2/co
lab_requirements.txt (line 89)) (2.4.1)
Requirement already satisfied: stack-data==0.6.2 in /home/jared/anaconda3/envs/CS635
3/lib/python3.10/site-packages (from -r /home/jared/CS-6353/Homeworks/assignment2/co
lab_requirements.txt (line 90)) (0.6.2)
Requirement already satisfied: terminado==0.17.1 in /home/jared/anaconda3/envs/CS635
3/lib/python3.10/site-packages (from -r /home/jared/CS-6353/Homeworks/assignment2/co
lab_requirements.txt (line 91)) (0.17.1)
Requirement already satisfied: tinycss2==1.2.1 in /home/jared/anaconda3/envs/CS6353/
lib/python3.10/site-packages (from -r /home/jared/CS-6353/Homeworks/assignment2/cola
b_requirements.txt (line 92)) (1.2.1)
Requirement already satisfied: tornado<=6.3.2 in /home/jared/anaconda3/envs/CS6353/l
ib/python3.10/site-packages (from -r /home/jared/CS-6353/Homeworks/assignment2/colab
_requirements.txt (line 93)) (6.3.2)
Requirement already satisfied: traitlets==5.9.0 in /home/jared/anaconda3/envs/CS635
3/lib/python3.10/site-packages (from -r /home/jared/CS-6353/Homeworks/assignment2/co
lab_requirements.txt (line 94)) (5.9.0)
Requirement already satisfied: tzdata==2023.3 in /home/jared/anaconda3/envs/CS6353/l
ib/python3.10/site-packages (from -r /home/jared/CS-6353/Homeworks/assignment2/colab
_requirements.txt (line 95)) (2023.3)
Requirement already satisfied: uri-template==1.3.0 in /home/jared/anaconda3/envs/CS6
353/lib/python3.10/site-packages (from -r /home/jared/CS-6353/Homeworks/assignment2/
colab_requirements.txt (line 96)) (1.3.0)
Requirement already satisfied: urllib3==2.0.4 in /home/jared/anaconda3/envs/CS6353/l
ib/python3.10/site-packages (from -r /home/jared/CS-6353/Homeworks/assignment2/colab
_requirements.txt (line 97)) (2.0.4)
Requirement already satisfied: wcwidth==0.2.6 in /home/jared/anaconda3/envs/CS6353/l
ib/python3.10/site-packages (from -r /home/jared/CS-6353/Homeworks/assignment2/colab
_requirements.txt (line 98)) (0.2.6)
Requirement already satisfied: webcolors==1.13 in /home/jared/anaconda3/envs/CS6353/
lib/python3.10/site-packages (from -r /home/jared/CS-6353/Homeworks/assignment2/cola
b_requirements.txt (line 99)) (1.13)
Requirement already satisfied: webencodings==0.5.1 in /home/jared/anaconda3/envs/CS6
353/lib/python3.10/site-packages (from -r /home/jared/CS-6353/Homeworks/assignment2/
colab_requirements.txt (line 100)) (0.5.1)
Requirement already satisfied: websocket-client==1.6.2 in /home/jared/anaconda3/env
s/CS6353/lib/python3.10/site-packages (from -r /home/jared/CS-6353/Homeworks/assignm
ent2/colab_requirements.txt (line 101)) (1.6.2)
Requirement already satisfied: exceptiongroup in /home/jared/anaconda3/envs/CS6353/l
ib/python3.10/site-packages (from anyio==3.7.1->-r /home/jared/CS-6353/Homeworks/ass
ignment2/colab_requirements.txt (line 1)) (1.3.0)
Requirement already satisfied: typing-extensions>=4.0.0 in /home/jared/anaconda3/env
s/CS6353/lib/python3.10/site-packages (from async-lru==2.0.4->-r /home/jared/CS-635
3/Homeworks/assignment2/colab_requirements.txt (line 7)) (4.15.0)
Requirement already satisfied: tomli in /home/jared/anaconda3/envs/CS6353/lib/python
3.10/site-packages (from jupyterlab==4.0.5->-r /home/jared/CS-6353/Homeworks/assignm
ent2/colab_requirements.txt (line 43)) (2.2.1)
Requirement already satisfied: ipython-genutils in /home/jared/anaconda3/envs/CS635
```

```
3/lib/python3.10/site-packages (from ipykernel<=5.5.6->-r /home/jared/CS-6353/Homeworks/assignment2/colab_requirements.txt (line 28)) (0.2.0)
Requirement already satisfied: setuptools>=18.5 in /home/jared/anaconda3/envs/CS6353/lib/python3.10/site-packages (from ipython<=7.34.0->-r /home/jared/CS-6353/Homeworks/assignment2/colab_requirements.txt (line 29)) (78.1.1)
Requirement already satisfied: entrypoints in /home/jared/anaconda3/envs/CS6353/lib/python3.10/site-packages (from jupyter_client<8.0->-r /home/jared/CS-6353/Homeworks/assignment2/colab_requirements.txt (line 39)) (0.4)
```

Multiclass Support Vector Machine exercise

Complete and hand in this completed worksheet (including its outputs and any supporting code outside of the worksheet) with your assignment submission. For more details see the [assignments page](#) on the course website.

In this exercise you will:

- implement a fully-vectorized **loss function** for the SVM
- implement the fully-vectorized expression for its **analytic gradient**
- **check your implementation** using numerical gradient
- use a validation set to **tune the learning rate and regularization** strength
- **optimize** the loss function with **SGD**
- **visualize** the final learned weights

```
In [2]: # Run some setup code for this notebook.
from __future__ import print_function
import random
import numpy as np
from cs6353.data_utils import load_CIFAR10
import matplotlib.pyplot as plt

# This is a bit of magic to make matplotlib figures appear inline in the
# notebook rather than in a new window.
%matplotlib inline
plt.rcParams['figure.figsize'] = (10.0, 8.0) # set default size of plots
plt.rcParams['image.interpolation'] = 'nearest'
plt.rcParams['image.cmap'] = 'gray'

# Some more magic so that the notebook will reload external python modules;
# see http://stackoverflow.com/questions/1907993/autoreload-of-modules-in-ipython
%load_ext autoreload
%autoreload 2
```

CIFAR-10 Data Loading and Preprocessing

```
In [3]: # Load the raw CIFAR-10 data.
cifar10_dir = 'cs6353/datasets/cifar-10-batches-py'
```

```
# Cleaning up variables to prevent loading data multiple times (which may cause mem
try:
    del X_train, y_train
    del X_test, y_test
    print('Clear previously loaded data.')
except:
    pass

X_train, y_train, X_test, y_test = load_CIFAR10(cifar10_dir)

# As a sanity check, we print out the size of the training and test data.
print('Training data shape: ', X_train.shape)
print('Training labels shape: ', y_train.shape)
print('Test data shape: ', X_test.shape)
print('Test labels shape: ', y_test.shape)
```

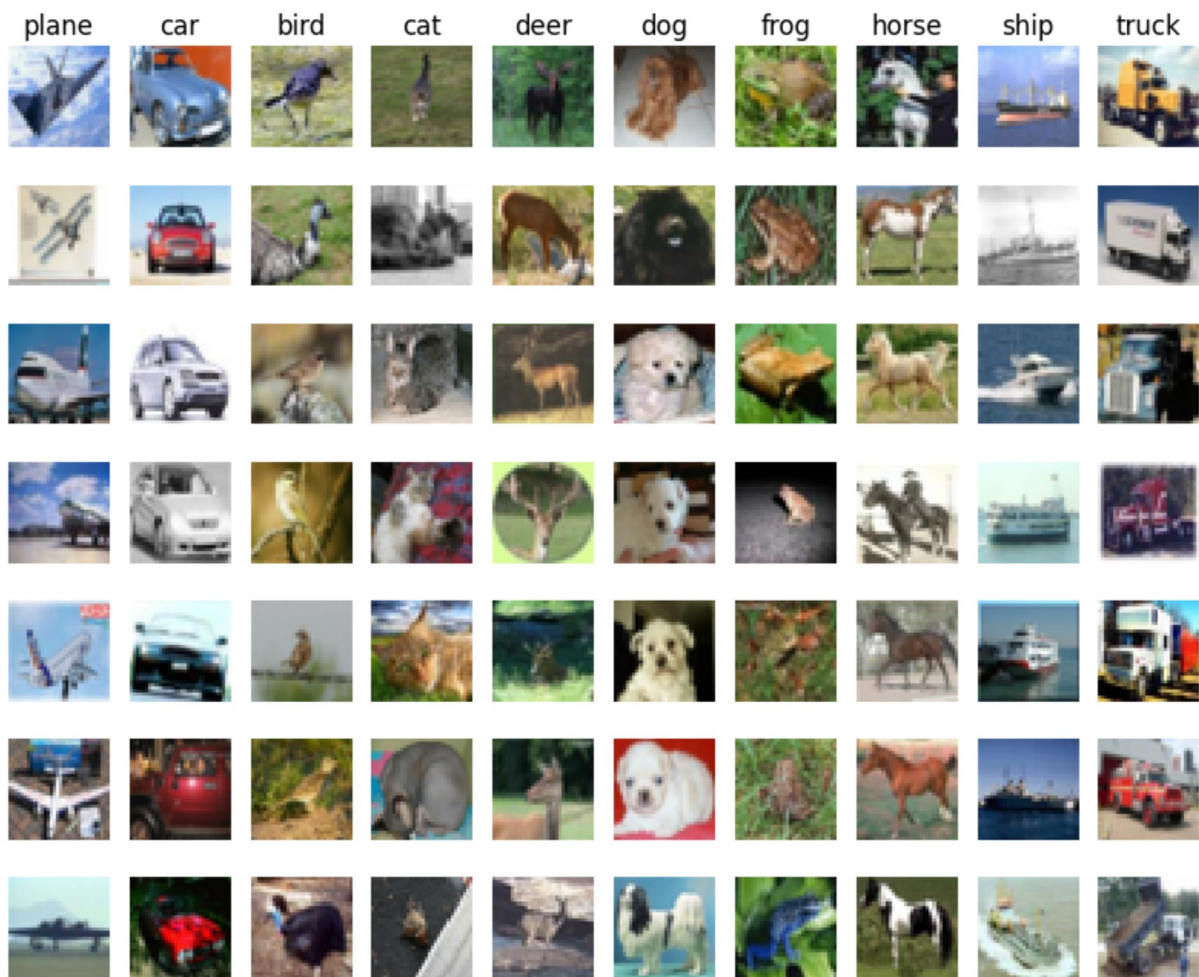
Training data shape: (50000, 32, 32, 3)

Training labels shape: (50000,)

Test data shape: (10000, 32, 32, 3)

Test labels shape: (10000,)

```
In [4]: # Visualize some examples from the dataset.
# We show a few examples of training images from each class.
classes = ['plane', 'car', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse', 'ship', '
num_classes = len(classes)
samples_per_class = 7
for y, cls in enumerate(classes):
    idxs = np.flatnonzero(y_train == y)
    idxs = np.random.choice(idxs, samples_per_class, replace=False)
    for i, idx in enumerate(idxs):
        plt_idx = i * num_classes + y + 1
        plt.subplot(samples_per_class, num_classes, plt_idx)
        plt.imshow(X_train[idx].astype('uint8'))
        plt.axis('off')
        if i == 0:
            plt.title(cls)
plt.show()
```



```
In [5]: # Split the data into train, val, and test sets. In addition we will
# create a small development set as a subset of the training data;
# we can use this for development so our code runs faster.
num_training = 49000
num_validation = 1000
num_test = 1000
num_dev = 500

# Our validation set will be num_validation points from the original
# training set.
mask = range(num_training, num_training + num_validation)
X_val = X_train[mask]
y_val = y_train[mask]

# Our training set will be the first num_train points from the original
# training set.
mask = range(num_training)
X_train = X_train[mask]
y_train = y_train[mask]

# We will also make a development set, which is a small subset of
# the training set.
mask = np.random.choice(num_training, num_dev, replace=False)
X_dev = X_train[mask]
y_dev = y_train[mask]
```

```
# We use the first num_test points of the original test set as our
# test set.
mask = range(num_test)
X_test = X_test[mask]
y_test = y_test[mask]

print('Train data shape: ', X_train.shape)
print('Train labels shape: ', y_train.shape)
print('Validation data shape: ', X_val.shape)
print('Validation labels shape: ', y_val.shape)
print('Test data shape: ', X_test.shape)
print('Test labels shape: ', y_test.shape)
```

```
Train data shape: (49000, 32, 32, 3)
Train labels shape: (49000,)
Validation data shape: (1000, 32, 32, 3)
Validation labels shape: (1000,)
Test data shape: (1000, 32, 32, 3)
Test labels shape: (1000,)
```

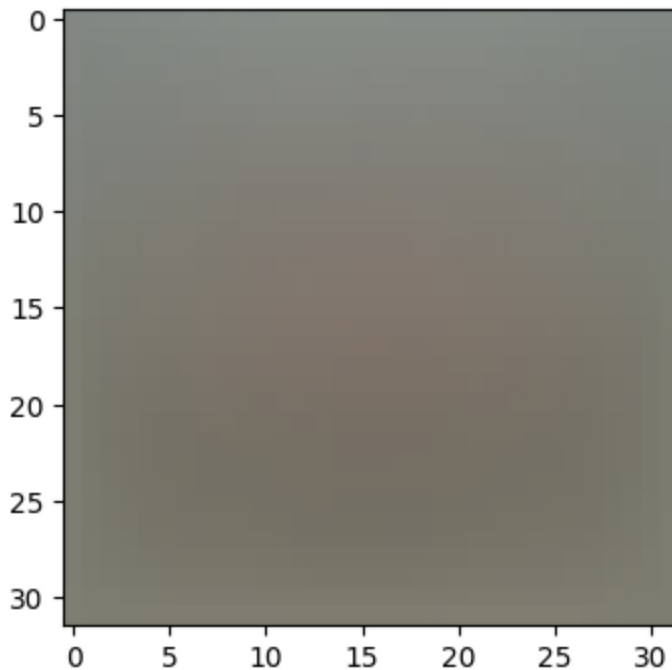
```
In [6]: # Preprocessing: reshape the image data into rows
X_train = np.reshape(X_train, (X_train.shape[0], -1))
X_val = np.reshape(X_val, (X_val.shape[0], -1))
X_test = np.reshape(X_test, (X_test.shape[0], -1))
X_dev = np.reshape(X_dev, (X_dev.shape[0], -1))

# As a sanity check, print out the shapes of the data
print('Training data shape: ', X_train.shape)
print('Validation data shape: ', X_val.shape)
print('Test data shape: ', X_test.shape)
print('dev data shape: ', X_dev.shape)
```

```
Training data shape: (49000, 3072)
Validation data shape: (1000, 3072)
Test data shape: (1000, 3072)
dev data shape: (500, 3072)
```

```
In [7]: # Preprocessing: subtract the mean image
# first: compute the image mean based on the training data
mean_image = np.mean(X_train, axis=0)
print(mean_image[:10]) # print a few of the elements
plt.figure(figsize=(4,4))
plt.imshow(mean_image.reshape((32,32,3)).astype('uint8')) # visualize the mean image
plt.show()
```

```
[130.64189796 135.98173469 132.47391837 130.05569388 135.34804082
 131.75402041 130.96055102 136.14328571 132.47636735 131.48467347]
```



```
In [8]: # second: subtract the mean image from train and test data
X_train -= mean_image
X_val -= mean_image
X_test -= mean_image
X_dev -= mean_image
```

```
In [9]: # third: append the bias dimension of ones (i.e. bias trick) so that our SVM
# only has to worry about optimizing a single weight matrix W.
X_train = np.hstack([X_train, np.ones((X_train.shape[0], 1))])
X_val = np.hstack([X_val, np.ones((X_val.shape[0], 1))])
X_test = np.hstack([X_test, np.ones((X_test.shape[0], 1))])
X_dev = np.hstack([X_dev, np.ones((X_dev.shape[0], 1))])

print(X_train.shape, X_val.shape, X_test.shape, X_dev.shape)
```

```
(49000, 3073) (1000, 3073) (1000, 3073) (500, 3073)
```

SVM Classifier

Your code for this section will all be written inside **cs6353/classifiers/linear_svm.py**.

As you can see, we have prefilled the function `svm_loss_naive` which uses for loops to evaluate the multiclass SVM loss function.

```
In [10]: # Evaluate the naive implementation of the loss we provided for you:
from cs6353.classifiers.linear_svm import svm_loss_naive, svm_loss_vectorized
import time

# generate a random SVM weight matrix of small numbers
W = np.random.randn(3073, 10) * 0.0001

loss, grad = svm_loss_naive(W, X_dev, y_dev, 0.000005)
```

```
print('loss: %f' % (loss, ))
print(grad.shape)
```

```
loss: 8.576507
(3073, 10)
```

The `grad` returned from the function above is right now all zero. Derive and implement the gradient for the SVM cost function and implement it inline inside the function

`svm_loss_naive`. You will find it helpful to interleave your new code inside the existing function.

To check that you have correctly implemented the gradient correctly, you can numerically estimate the gradient of the loss function and compare the numeric estimate to the gradient that you computed. We have provided code that does this for you:

```
In [11]: # Once you've implemented the gradient, recompute it with the code below
# and gradient check it with the function we provided for you

# Compute the loss and its gradient at W.
loss, grad = svm_loss_naive(W, X_dev, y_dev, 0.0)

# Numerically compute the gradient along several randomly chosen dimensions, and
# compare them with your analytically computed gradient. The numbers should match
# almost exactly along all dimensions.
from cs6353.gradient_check import grad_check_sparse
f = lambda w: svm_loss_naive(w, X_dev, y_dev, 0.0)[0]
grad_numerical = grad_check_sparse(f, W, grad)

# do the gradient check once again with regularization turned on
# you didn't forget the regularization gradient did you?
loss, grad = svm_loss_naive(W, X_dev, y_dev, 5e1)
f = lambda w: svm_loss_naive(w, X_dev, y_dev, 5e1)[0]
grad_numerical = grad_check_sparse(f, W, grad)
```

```
numerical: 9.170448 analytic: 9.170448, relative error: 3.632114e-11
numerical: -1.317196 analytic: -1.317196, relative error: 3.358994e-10
numerical: -19.008556 analytic: -19.008556, relative error: 1.117134e-11
numerical: -10.231174 analytic: -10.231174, relative error: 5.826754e-13
numerical: 7.746033 analytic: 7.746033, relative error: 1.451390e-11
numerical: -12.672925 analytic: -12.672925, relative error: 9.044503e-12
numerical: -24.129162 analytic: -24.129162, relative error: 5.085210e-12
numerical: 25.230402 analytic: 25.230402, relative error: 1.429772e-11
numerical: -8.017342 analytic: -8.017342, relative error: 5.374918e-11
numerical: -20.279785 analytic: -20.279785, relative error: 1.504664e-12
numerical: 7.253526 analytic: 7.235519, relative error: 1.242847e-03
numerical: 22.750309 analytic: 22.757072, relative error: 1.485985e-04
numerical: 30.202599 analytic: 30.202371, relative error: 3.785833e-06
numerical: 10.458382 analytic: 10.445763, relative error: 6.036583e-04
numerical: 3.491665 analytic: 3.487161, relative error: 6.453447e-04
numerical: 1.631983 analytic: 1.629529, relative error: 7.526347e-04
numerical: 2.070403 analytic: 2.086633, relative error: 3.904142e-03
numerical: -7.223279 analytic: -7.237905, relative error: 1.011431e-03
numerical: -5.502103 analytic: -5.508539, relative error: 5.844731e-04
numerical: 6.067386 analytic: 6.069585, relative error: 1.812352e-04
```

Inline Question 1:

It is possible that once in a while a dimension in the gradient check will not match exactly (i). What could such a discrepancy be caused by? (ii) Is it a reason for concern? (iii) What is a simple example in one dimension where a gradient check could fail? (iv) How would change the margin affect of the frequency of this happening? (v) *Hint: the SVM loss function is not strictly speaking differentiable*

Your Answer:

- Reponse:
- (i): A gradient check can not match exactly if even one of the dimensions isn't differentiable
- (ii): It isn't really reason for concern because we have sub-gradients and we can work around it. In practice things like ReLu end up being better because of the fact that they are differentiable.
- (iii / iv): Imagine a 1 dimensional line. We can think of the equation of $\max(0, a_1 - b_1 + \delta)$. We know that our equation is non differentiable at 0 (thats where the max function prodces a break). No matter what our values of a_1 and b_1 are the margin term pushes to right on the line. We want to be further to the right of the line to avoid being at the non differentiable point. For example if $a_1 - a_2 = 0$ and our margin is 1 we just got ourselves out of the non-differentiable region. This same example works in the non 1d case as well. We want to be away from the non-differentiable region and the higher margin helps with that. Putting the equation in a graphing software and wiggling around our delta also proves this.

```
In [12]: # Next implement the function svm_loss_vectorized; for now only compute the loss;
# we will implement the gradient in a moment.
tic = time.time()
loss_naive, grad_naive = svm_loss_naive(W, X_dev, y_dev, 0.000005)
toc = time.time()
print('Naive loss: %e computed in %fs' % (loss_naive, toc - tic))

from cs6353.classifiers.linear_svm import svm_loss_vectorized
tic = time.time()
loss_vectorized, _ = svm_loss_vectorized(W, X_dev, y_dev, 0.000005)
toc = time.time()
print('Vectorized loss: %e computed in %fs' % (loss_vectorized, toc - tic))

# The losses should match but your vectorized implementation should be much faster.
print('difference: %f' % (loss_naive - loss_vectorized))
```

```
Naive loss: 8.576507e+00 computed in 0.383843s
Vectorized loss: 8.576507e+00 computed in 0.003682s
difference: 0.000000
```

```
In [13]: # Complete the implementation of svm_loss_vectorized, and compute the gradient
# of the loss function in a vectorized way.
```

```
# The naive implementation and the vectorized implementation should match, but
# the vectorized version should still be much faster.
tic = time.time()
_, grad_naive = svm_loss_naive(W, X_dev, y_dev, 0.000005)
toc = time.time()
print('Naive loss and gradient: computed in %fs' % (toc - tic))

tic = time.time()
_, grad_vectorized = svm_loss_vectorized(W, X_dev, y_dev, 0.000005)
toc = time.time()
print('Vectorized loss and gradient: computed in %fs' % (toc - tic))

# The loss is a single number, so it is easy to compare the values computed
# by the two implementations. The gradient on the other hand is a matrix, so
# we use the Frobenius norm to compare them.
difference = np.linalg.norm(grad_naive - grad_vectorized, ord='fro')
print('difference: %f' % difference)
```

Naive loss and gradient: computed in 0.432430s

Vectorized loss and gradient: computed in 0.003753s

difference: 0.000000

Stochastic Gradient Descent

We now have vectorized and efficient expressions for the loss, the gradient and our gradient matches the numerical gradient. We are therefore ready to do SGD to minimize the loss.

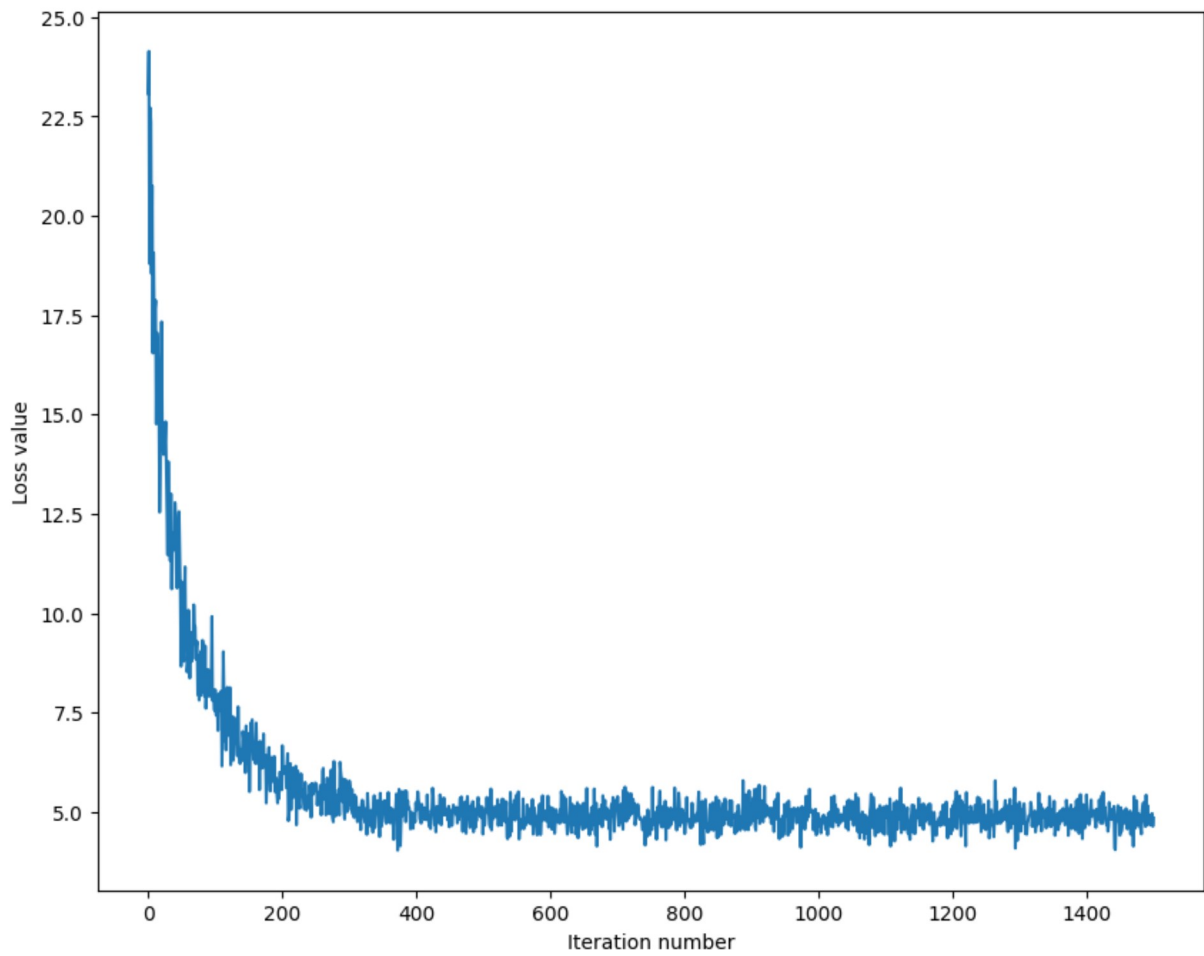
Your code for this part will be written inside `cs6353/classifiers/`

`linear_classifier.py`.

```
In [14]: # In the file linear_classifier.py, implement SGD in the function
# LinearClassifier.train() and then run it with the code below.
from cs6353.classifiers import LinearSVM
svm = LinearSVM()
tic = time.time()
loss_hist = svm.train(X_train, y_train, learning_rate=1e-7, reg=2.5e4,
                      num_iters=1500, verbose=True)
toc = time.time()
print('That took %fs' % (toc - tic))
```

```
iteration 0 / 1500: loss 23.075952
iteration 100 / 1500: loss 8.075309
iteration 200 / 1500: loss 6.669199
iteration 300 / 1500: loss 5.782761
iteration 400 / 1500: loss 5.052676
iteration 500 / 1500: loss 4.855971
iteration 600 / 1500: loss 4.999882
iteration 700 / 1500: loss 5.389075
iteration 800 / 1500: loss 4.451210
iteration 900 / 1500: loss 5.212317
iteration 1000 / 1500: loss 4.853245
iteration 1100 / 1500: loss 4.873452
iteration 1200 / 1500: loss 5.165168
iteration 1300 / 1500: loss 5.215363
iteration 1400 / 1500: loss 5.109425
That took 6.797943s
```

```
In [15]: # A useful debugging strategy is to plot the loss as a function of
# iteration number:
plt.plot(loss_hist)
plt.xlabel('Iteration number')
plt.ylabel('Loss value')
plt.show()
```



```
In [16]: # Write the LinearSVM.predict function and evaluate the performance on both the
# training and validation set
y_train_pred = svm.predict(X_train)
print('training accuracy: %f' % (np.mean(y_train == y_train_pred), ))
y_val_pred = svm.predict(X_val)
print('validation accuracy: %f' % (np.mean(y_val == y_val_pred), ))
```

```
training accuracy: 0.365939
validation accuracy: 0.391000
```

```
In [17]: # Use the validation set to tune hyperparameters (regularization strength and
# learning rate). You should experiment with different ranges for the learning
# rates and regularization strengths; if you are careful you should be able to
# get a classification accuracy of about 0.4 on the validation set.
# learning_rates = [1e-7, 5e-5]
# regularization_strengths = [2.5e4, 5e4]

learning_rates = np.logspace(-8,-3,7)
regularization_strengths = np.logspace(2,6,7)

# learning_rates = [1e-8, 1e-7, 1e-6, 1e-5, 1e-4]
# regularization_strengths = [1e+5, 1e+6, 1e+7, 1e+8, 1e+9]

print("possible learning rates: " + str(learning_rates))
print("possible regularization strengths: " + str(regularization_strengths))

# results is dictionary mapping tuples of the form
```

```

# (learning_rate, regularization_strength) to tuples of the form
# (training_accuracy, validation_accuracy). The accuracy is simply the fraction
# of data points that are correctly classified.
results = {}
best_val = -1 # The highest validation accuracy that we have seen so far.
best_svm = None # The LinearSVM object that achieved the highest validation rate.

#####
# TODO:
# Write code that chooses the best hyperparameters by tuning on the validation
# set. For each combination of hyperparameters, train a linear SVM on the
# training set, compute its accuracy on the training and validation sets, and
# store these numbers in the results dictionary. In addition, store the best
# validation accuracy in best_val and the LinearSVM object that achieves this
# accuracy in best_svm.
#
# Hint: You should use a small value for num_iters as you develop your
# validation code so that the SVMs don't take much time to train; once you are
# confident that your validation code works, you should rerun the validation
# code with a larger value for num_iters.
#####
# Your code
for cur_lr in learning_rates:
    for r in regularization_strengths:
        curSVM = LinearSVM()
        curSVM.train(X_train, y_train, cur_lr, r, num_iters=1000, batch_size=200)

        # Get training accuracy
        y_train_pred = curSVM.predict(X_train)
        train_accuracy = np.mean(y_train == y_train_pred)

        # Get validation accuracy
        y_val_pred = curSVM.predict(X_val)
        validation_accuracy = np.mean(y_val == y_val_pred)

        results[(cur_lr, r)] = (train_accuracy, validation_accuracy)
        if validation_accuracy > best_val:
            best_val = validation_accuracy
            best_svm = curSVM
#####
#                                     END OF YOUR CODE                                     #
#####

# Print out results.
for lr, reg in sorted(results):
    train_accuracy, val_accuracy = results[(lr, reg)]
    print('lr %e reg %e train accuracy: %f val accuracy: %f' % (
        lr, reg, train_accuracy, val_accuracy))

print('best validation accuracy achieved during cross-validation: %f' % best_val)

```

```
possible learning rates: [1.00000000e-08 6.81292069e-08 4.64158883e-07 3.16227766e-06
2.15443469e-05 1.46779927e-04 1.00000000e-03]
possible regularization strengths: [1.00000000e+02 4.64158883e+02 2.15443469e+03 1.00000000e+04
4.64158883e+04 2.15443469e+05 1.00000000e+06]
/home/jared/CS-6353/Homeworks/assignment2/cs6353/classifiers/linear_svm.py:106: RuntimeWarning: overflow encountered in multiply
    dW += 2 * W * reg
/home/jared/CS-6353/Homeworks/assignment2/cs6353/classifiers/linear_svm.py:80: RuntimeWarning: invalid value encountered in subtract
    margins = scores - correct_class_scores.reshape(-1,1) + 1
/home/jared/CS-6353/Homeworks/assignment2/cs6353/classifiers/linear_svm.py:78: RuntimeWarning: invalid value encountered in matmul
    scores = X @ W
/home/jared/anaconda3/envs/CS6353/lib/python3.10/site-packages/numpy/core/fromnumeric.py:86: RuntimeWarning: overflow encountered in reduce
    return ufunc.reduce(obj, axis, dtype, out, **passkwargs)
/home/jared/CS-6353/Homeworks/assignment2/cs6353/classifiers/linear_svm.py:80: RuntimeWarning: overflow encountered in subtract
    margins = scores - correct_class_scores.reshape(-1,1) + 1
/home/jared/CS-6353/Homeworks/assignment2/cs6353/classifiers/linear_svm.py:78: RuntimeWarning: overflow encountered in matmul
    scores = X @ W
```

```

lr 1.000000e-08 reg 1.000000e+02 train accuracy: 0.209633 val accuracy: 0.222000
lr 1.000000e-08 reg 4.641589e+02 train accuracy: 0.205429 val accuracy: 0.188000
lr 1.000000e-08 reg 2.154435e+03 train accuracy: 0.197102 val accuracy: 0.199000
lr 1.000000e-08 reg 1.000000e+04 train accuracy: 0.215673 val accuracy: 0.223000
lr 1.000000e-08 reg 4.641589e+04 train accuracy: 0.236408 val accuracy: 0.255000
lr 1.000000e-08 reg 2.154435e+05 train accuracy: 0.331184 val accuracy: 0.346000
lr 1.000000e-08 reg 1.000000e+06 train accuracy: 0.286000 val accuracy: 0.297000
lr 6.812921e-08 reg 1.000000e+02 train accuracy: 0.282265 val accuracy: 0.311000
lr 6.812921e-08 reg 4.641589e+02 train accuracy: 0.282000 val accuracy: 0.282000
lr 6.812921e-08 reg 2.154435e+03 train accuracy: 0.287673 val accuracy: 0.302000
lr 6.812921e-08 reg 1.000000e+04 train accuracy: 0.345163 val accuracy: 0.345000
lr 6.812921e-08 reg 4.641589e+04 train accuracy: 0.360306 val accuracy: 0.369000
lr 6.812921e-08 reg 2.154435e+05 train accuracy: 0.323694 val accuracy: 0.326000
lr 6.812921e-08 reg 1.000000e+06 train accuracy: 0.299041 val accuracy: 0.301000
lr 4.641589e-07 reg 1.000000e+02 train accuracy: 0.341776 val accuracy: 0.345000
lr 4.641589e-07 reg 4.641589e+02 train accuracy: 0.355959 val accuracy: 0.337000
lr 4.641589e-07 reg 2.154435e+03 train accuracy: 0.383245 val accuracy: 0.388000
lr 4.641589e-07 reg 1.000000e+04 train accuracy: 0.372000 val accuracy: 0.386000
lr 4.641589e-07 reg 4.641589e+04 train accuracy: 0.328959 val accuracy: 0.361000
lr 4.641589e-07 reg 2.154435e+05 train accuracy: 0.287000 val accuracy: 0.305000
lr 4.641589e-07 reg 1.000000e+06 train accuracy: 0.204163 val accuracy: 0.207000
lr 3.162278e-06 reg 1.000000e+02 train accuracy: 0.308653 val accuracy: 0.318000
lr 3.162278e-06 reg 4.641589e+02 train accuracy: 0.299265 val accuracy: 0.302000
lr 3.162278e-06 reg 2.154435e+03 train accuracy: 0.237143 val accuracy: 0.227000
lr 3.162278e-06 reg 1.000000e+04 train accuracy: 0.252653 val accuracy: 0.249000
lr 3.162278e-06 reg 4.641589e+04 train accuracy: 0.197184 val accuracy: 0.201000
lr 3.162278e-06 reg 2.154435e+05 train accuracy: 0.131857 val accuracy: 0.115000
lr 3.162278e-06 reg 1.000000e+06 train accuracy: 0.100265 val accuracy: 0.087000
lr 2.154435e-05 reg 1.000000e+02 train accuracy: 0.255429 val accuracy: 0.261000
lr 2.154435e-05 reg 4.641589e+02 train accuracy: 0.263673 val accuracy: 0.255000
lr 2.154435e-05 reg 2.154435e+03 train accuracy: 0.215980 val accuracy: 0.228000
lr 2.154435e-05 reg 1.000000e+04 train accuracy: 0.169939 val accuracy: 0.167000
lr 2.154435e-05 reg 4.641589e+04 train accuracy: 0.051714 val accuracy: 0.048000
lr 2.154435e-05 reg 2.154435e+05 train accuracy: 0.100265 val accuracy: 0.087000
lr 2.154435e-05 reg 1.000000e+06 train accuracy: 0.100265 val accuracy: 0.087000
lr 1.467799e-04 reg 1.000000e+02 train accuracy: 0.223469 val accuracy: 0.227000
lr 1.467799e-04 reg 4.641589e+02 train accuracy: 0.176061 val accuracy: 0.190000
lr 1.467799e-04 reg 2.154435e+03 train accuracy: 0.156490 val accuracy: 0.157000
lr 1.467799e-04 reg 1.000000e+04 train accuracy: 0.045755 val accuracy: 0.040000
lr 1.467799e-04 reg 4.641589e+04 train accuracy: 0.100265 val accuracy: 0.087000
lr 1.467799e-04 reg 2.154435e+05 train accuracy: 0.100265 val accuracy: 0.087000
lr 1.467799e-04 reg 1.000000e+06 train accuracy: 0.100265 val accuracy: 0.087000
lr 1.000000e-03 reg 1.000000e+02 train accuracy: 0.198265 val accuracy: 0.178000
lr 1.000000e-03 reg 4.641589e+02 train accuracy: 0.167327 val accuracy: 0.167000
lr 1.000000e-03 reg 2.154435e+03 train accuracy: 0.100265 val accuracy: 0.087000
lr 1.000000e-03 reg 1.000000e+04 train accuracy: 0.100265 val accuracy: 0.087000
lr 1.000000e-03 reg 4.641589e+04 train accuracy: 0.100265 val accuracy: 0.087000
lr 1.000000e-03 reg 2.154435e+05 train accuracy: 0.100265 val accuracy: 0.087000
lr 1.000000e-03 reg 1.000000e+06 train accuracy: 0.100265 val accuracy: 0.087000
best validation accuracy achieved during cross-validation: 0.388000

```

```

In [18]: # Visualize the cross-validation results
import math
x_scatter = [math.log10(x[0]) for x in results]
y_scatter = [math.log10(x[1]) for x in results]

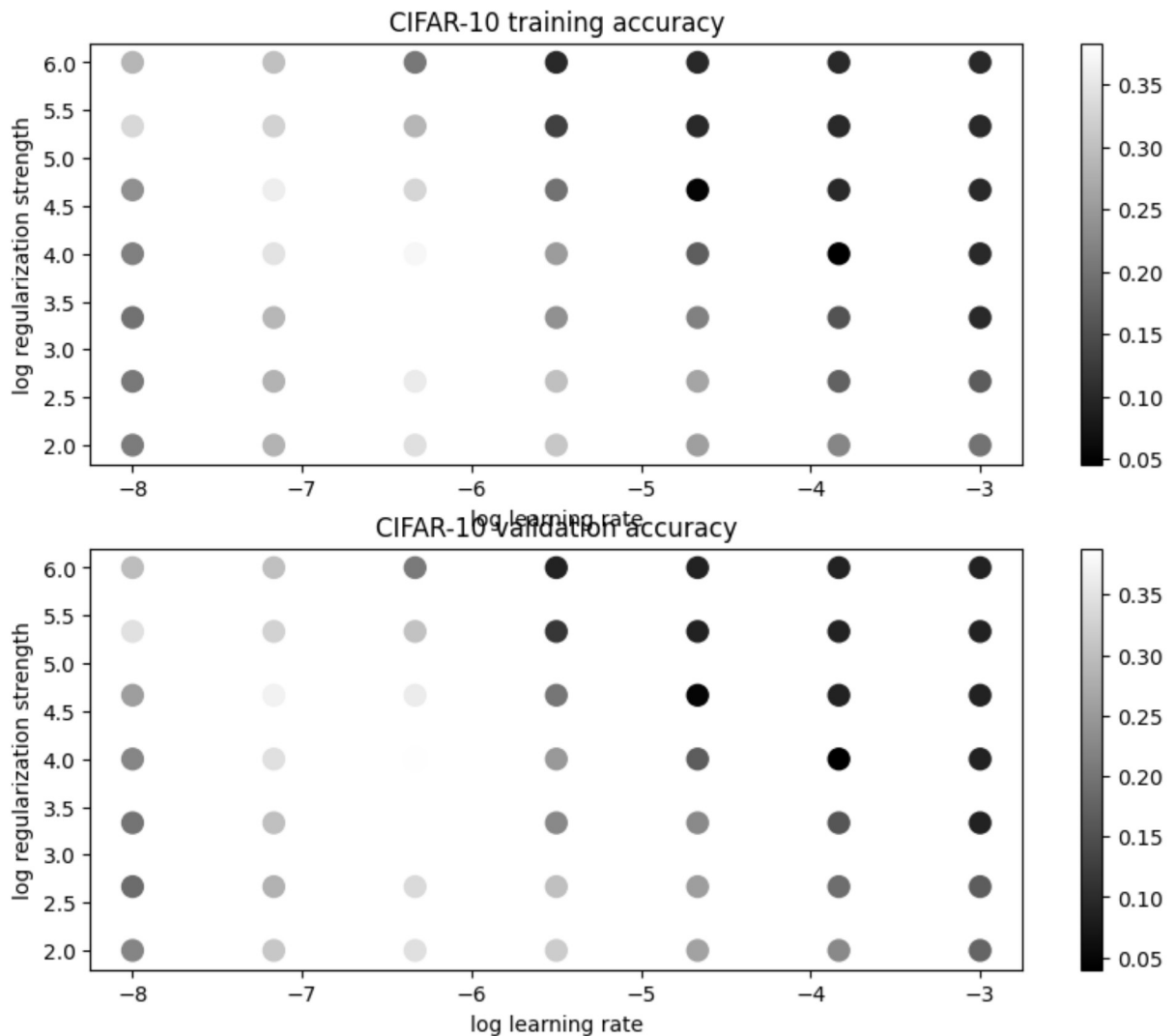
```

```

# plot training accuracy
marker_size = 100
colors = [results[x][0] for x in results]
plt.subplot(2, 1, 1)
plt.scatter(x_scatter, y_scatter, marker_size, c=colors)
plt.colorbar()
plt.xlabel('log learning rate')
plt.ylabel('log regularization strength')
plt.title('CIFAR-10 training accuracy')

# plot validation accuracy
colors = [results[x][1] for x in results] # default size of markers is 20
plt.subplot(2, 1, 2)
plt.scatter(x_scatter, y_scatter, marker_size, c=colors)
plt.colorbar()
plt.xlabel('log learning rate')
plt.ylabel('log regularization strength')
plt.title('CIFAR-10 validation accuracy')
plt.show()

```



```

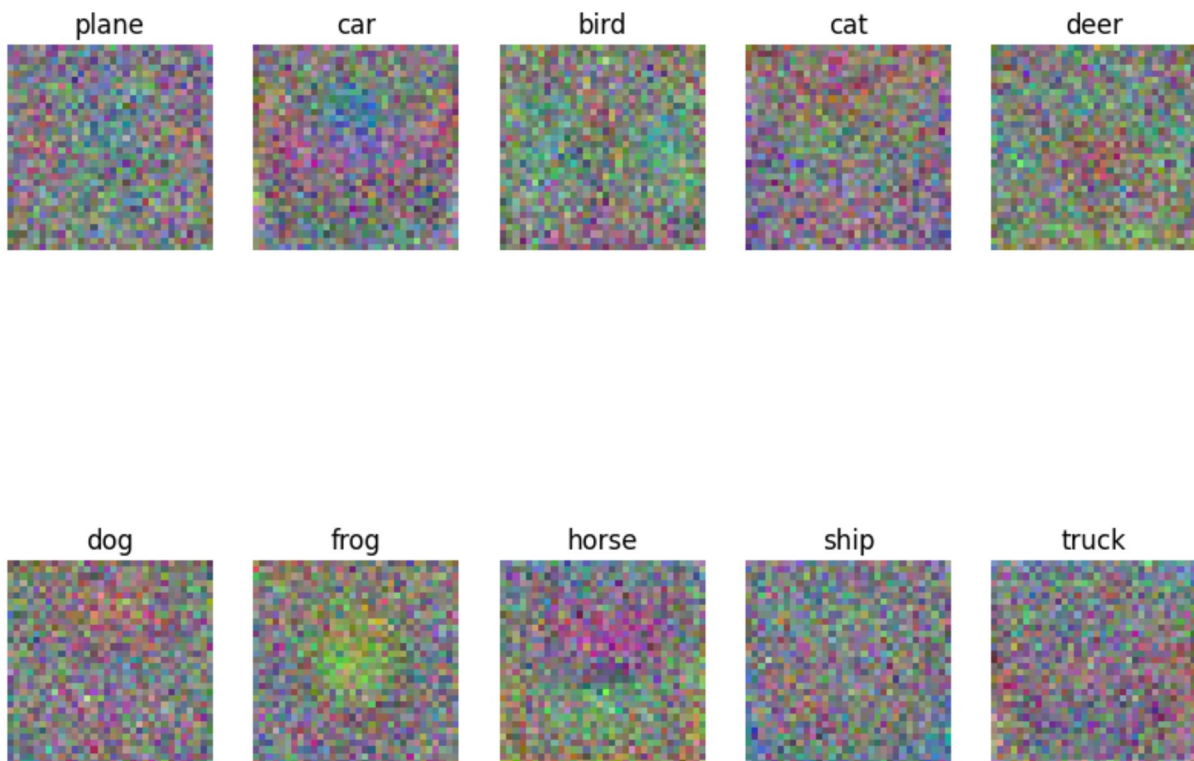
In [19]: # Evaluate the best svm on test set
y_test_pred = best_svm.predict(X_test)
test_accuracy = np.mean(y_test == y_test_pred)

```

```
print('linear SVM on raw pixels final test set accuracy: %f' % test_accuracy)
```

linear SVM on raw pixels final test set accuracy: 0.371000

```
In [20]: # Visualize the learned weights for each class.  
# Depending on your choice of learning rate and regularization strength, these may  
# or may not be nice to look at.  
w = best_svm.W[:-1,:] # strip out the bias  
w = w.reshape(32, 32, 3, 10)  
w_min, w_max = np.min(w), np.max(w)  
classes = ['plane', 'car', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse', 'ship', 'truck']  
for i in range(10):  
    plt.subplot(2, 5, i + 1)  
  
    # Rescale the weights to be between 0 and 255  
    wimg = 255.0 * (w[:, :, :, i].squeeze() - w_min) / (w_max - w_min)  
    plt.imshow(wimg.astype('uint8'))  
    plt.axis('off')  
    plt.title(classes[i])
```



Inline question 2:

(i) Describe what your visualized SVM weights look like, (ii) and offer a brief explanation for why they look the way that they do.

Your answer:

- (i) We notice that each visualized weight looks somewhat like the class it represents. I think that this is most clearly seen in the horse image where you can see what looks to be the outline of a horse.

- (ii) This makes a lot of sense when we think about what we just did. We tried to minimize the svm loss overall examples such that $W.T @ X$ produced the smallest amount of loss. When we take a new image with features and dot them with these images we can use that result to classify the image. I like to think about these blocks as a "template" for that specific class. It was very interesting to see this result and it was even cooler to see how it intuitively made sense.